

COMPLEX ENGINEERING PROBLEM

TITLE

API TESTING AND ERROR HANDLING

Vishwajit Mohol [92400120438]

Yagnik Bhojak [92400120585]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

5EV7

BATCH

A

COMPLEX ENGINEERING PROBLEM – REPORT

RUBRICS FOR COMPLEX ENGINEERING PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security.	3 Marks: Good system design and technical discussion with minor omissions.	1-2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
Total						10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	API TESTING AND ERROR HANDLING							

Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand the complete solution and should be able to explain their contribution during evaluation.

COMPLEX ENGINEERING PROBLEM - REPORT

TITLE: API TESTING AND ERROR HANDLING

Course Name: OPEN SOURCE TECHNOLOGIES (01CE0526)

Co's Mapped – Evaluate software reliability using open-source testing frameworks and tools.

SDG's Mapped – SDG 9 Industry, Innovation and Infrastructure

1. Aim

To test a CRUD-based REST API using Postman with valid, invalid, and missing inputs, verify HTTP status codes, response data, authentication, and error messages, and identify possible improvements in API error handling.

2. Problem Statement

- Test a provided CRUD API using valid, invalid, and missing input values. Verify HTTP status codes, error messages, and response data; then recommend improvements.
- The API provides operations for authentication and product management through REST-based endpoints.
- The testing focuses on verifying whether the API behaves correctly for successful requests as well as invalid, incomplete, and unauthorized requests.

3. Problem Analysis

a) What is the engineering problem?

A REST API must correctly process valid requests while rejecting invalid, incomplete, and unauthorized requests. The engineering problem is to systematically test the API, verify its HTTP status codes and response data, analyze its error messages, and identify improvements that can make API behavior more reliable and understandable for client applications.

b) Why is this problem complex?

- Requires multiple engineering concepts
- Requires analysis and decision making
- Involves design constraints
- Requires use of engineering tools
- Requires performance/security/safety consideration

The complexity arises because an API does not only need to return the correct result for successful requests. It must also handle invalid input, missing fields, unauthorized access, nonexistent resources, and other failure conditions in a consistent manner.

4. Knowledge and Tools Required

4.1 Open Source Technology Selection

- Postman was selected as the primary API testing tool because it provides a practical environment for sending HTTP requests, configuring authentication, supplying JSON request bodies, and inspecting HTTP status codes and response data. It is directly relevant to API testing and supports systematic testing of successful and erroneous API requests.
- The REST API architecture was used as the system under test because it provides CRUD operations through standard HTTP methods.
- JSON was used for structured request and response data, while HTTP status codes were used to determine the outcome of each request.
- The selected tools are appropriate for this problem because the objective is to evaluate API reliability through functional, negative-input, authentication, and error-handling tests.

COMPLEX ENGINEERING PROBLEM - REPORT

Engineering concepts used:

1. REST API architecture
2. HTTP methods and status code
3. CRUD operations
4. JSON request and response handling
5. Authentication and Protected API Access
6. API error handling

Technologies/Software used:

1. Postman
2. REST API
3. HTTP
4. JSON
5. Local Server

Postman was used to send API requests and inspect the returned status codes, response bodies, authentication behavior, and error messages. The API was hosted locally at <http://localhost:3000>.

5. Proposed Solution Design

The proposed testing system consists of Postman as the API client and the REST API as the system under test. Requests are sent from Postman to the locally hosted API. The API processes authentication, validation, and CRUD operations and returns an HTTP response containing a status code and JSON response data.

Block Diagram / Architecture Diagram:

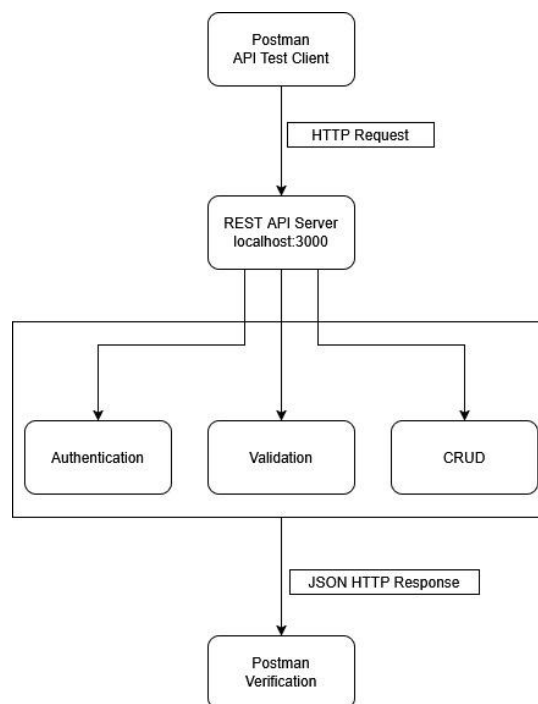


Figure 1 — System Architecture Diagram

COMPLEX ENGINEERING PROBLEM - REPORT

- Postman: Acts as the API testing client. It is used to construct HTTP requests, provide request bodies and authentication credentials, send requests, and inspect the responses.
- REST API Server: It is the system under test. It provides authentication and Product CRUD endpoints.
- Authentication Layer: It is tested using valid credentials, invalid credentials, missing credentials, and protected API requests.
- Product API: It provides operations to retrieve, create, update, and delete products.
- HTTP Layer: HTTP methods such as GET, POST, PUT, and DELETE are used to perform the required operations. The API communicates the result using HTTP status codes.
- JSON Layer: JSON is used for request and response data. This allows the client to provide structured input and inspect structured output and error messages.

Brief Working:

The API testing process begins by sending HTTP requests from Postman to the locally hosted REST API. The API first processes authentication and validates the incoming request. For valid requests, the required CRUD operation is performed and the API returns the appropriate HTTP status code and JSON response. For invalid, incomplete, or unauthorized requests, the API rejects the request and returns an error status code with a descriptive message. The returned status code and response body are then inspected in Postman to determine whether the API behaved as expected. Additional GET requests are used to verify the state of resources after create, update, and delete operations.

The main endpoints used during testing were:

Endpoint	HTTP Method	Purpose
/api/auth/login	POST	Authenticate a user
/api/products	GET	Retrieve all products
/api/products/:id	GET	Retrieve a product by ID
/api/products	POST	Create a new product
/api/products/:id	PUT	Update an existing product
/api/products/:id	DELETE	Delete a product

The Product object used during testing contains the fields id, name, price, and category.

Technical Analysis

- Deployment: The API was deployed locally on localhost:3000 for testing. This simplified the testing environment and allowed requests to be executed without dependence on an external server. In a production environment, the API would require a properly configured server, HTTPS, environment-specific configuration, logging, and monitoring.
- Version Control: The API testing artifacts and report can be maintained using Git so that changes to API requests, test collections, and documentation can be tracked. Git is an open-source version control system included within the course's open-source technology scope.
- Scalability: The current testing setup uses a single local API server and is therefore not designed for high traffic. For a production system, scalability could be improved by deploying multiple API instances behind a reverse proxy or load balancer.
- Reliability: Reliability was evaluated through positive and negative API test cases. Successful CRUD operations were verified after execution, while invalid input, authentication failures, and non-existent resources were explicitly tested.
- Security: Authentication was tested using valid and invalid credentials, and protected operations were tested without authentication. Input validation was also tested using invalid and missing values. In a production system, HTTPS, secure token management, rate limiting, authorization controls, and centralized security logging would also be required.

COMPLEX ENGINEERING PROBLEM - REPORT

6. Implementation / Procedure

Step 1: Start and Verify the API Server

The REST API was started locally on port 3000.

The health endpoint was then tested using: `GET http://localhost:3000/api/health`

The API returned `200 OK`, confirming that the server was running and ready for testing.

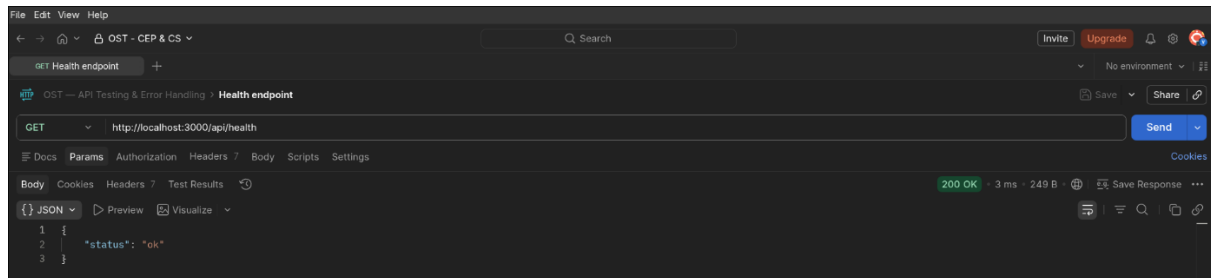


Figure 2: API Health Check Returning 200 OK

Step 2: Test Authentication

Authentication was tested using valid credentials, invalid credentials, and missing input.

A valid login request returned `200 OK` and an authentication token.

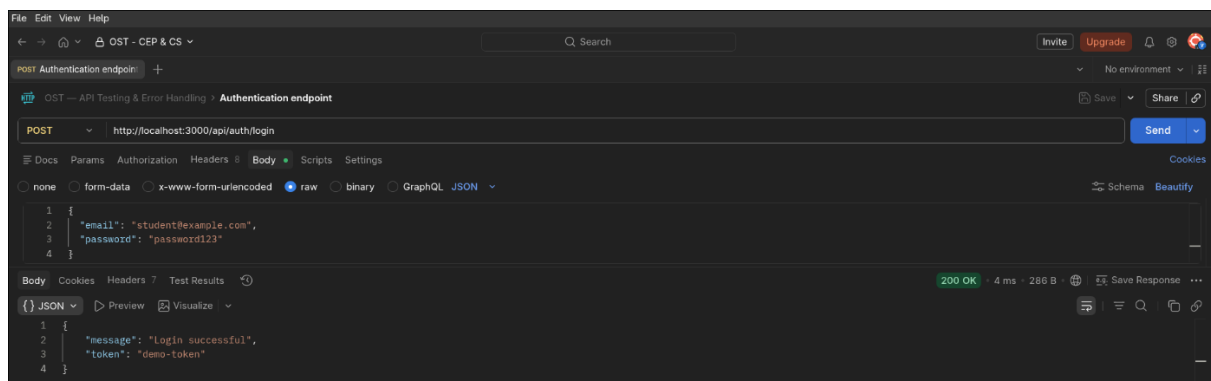


Figure 3: Successful Authentication Using Valid Credentials

Invalid credentials were then tested. The API returned `401 Unauthorized`, correctly rejecting the request.

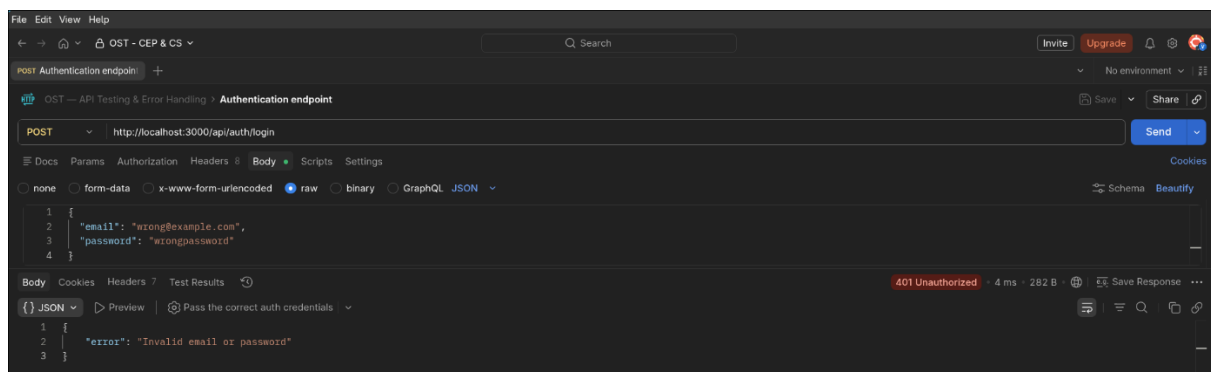


Figure 4: Authentication Failure with Invalid Credentials

A request with the password field missing returned `400 Bad Request`.

COMPLEX ENGINEERING PROBLEM - REPORT

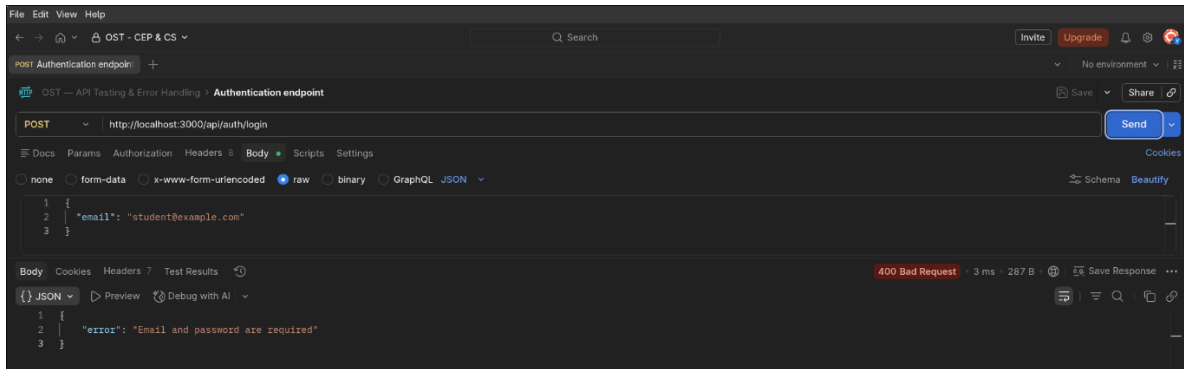


Figure 5: Authentication Request with Missing Password

Step 3: Retrieve Products

The GET endpoint was used to retrieve the available products: GET <http://localhost:3000/api/products>
The API returned the product collection with 200 OK

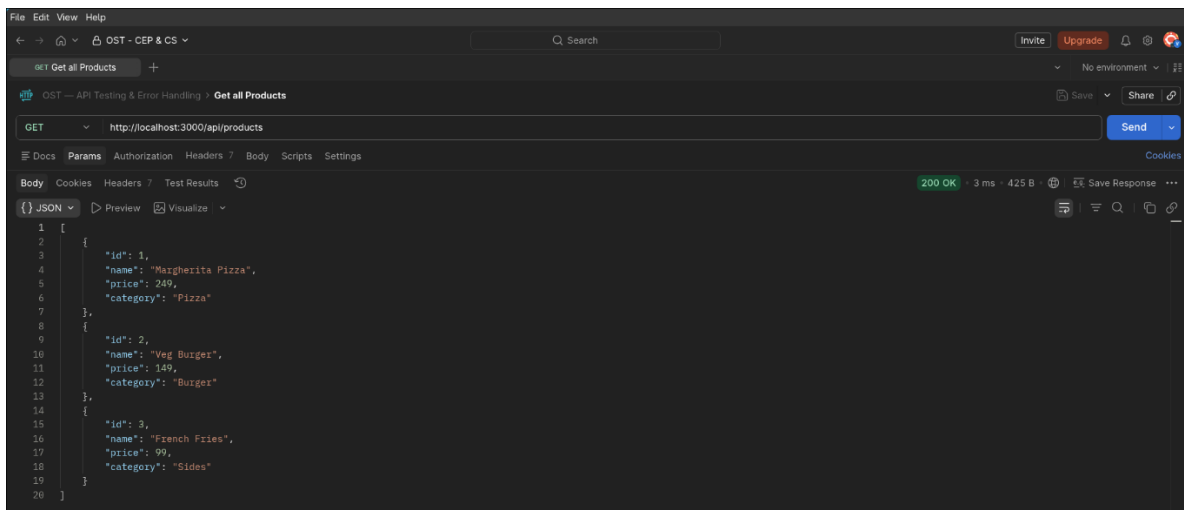


Figure 6: Retrieving Products Using GET Request

Step 4: Test Unauthorized Product Creation

A POST request was sent to the product endpoint without authentication.
The API returned 401 Unauthorized, demonstrating that the protected operation rejected an unauthenticated request.

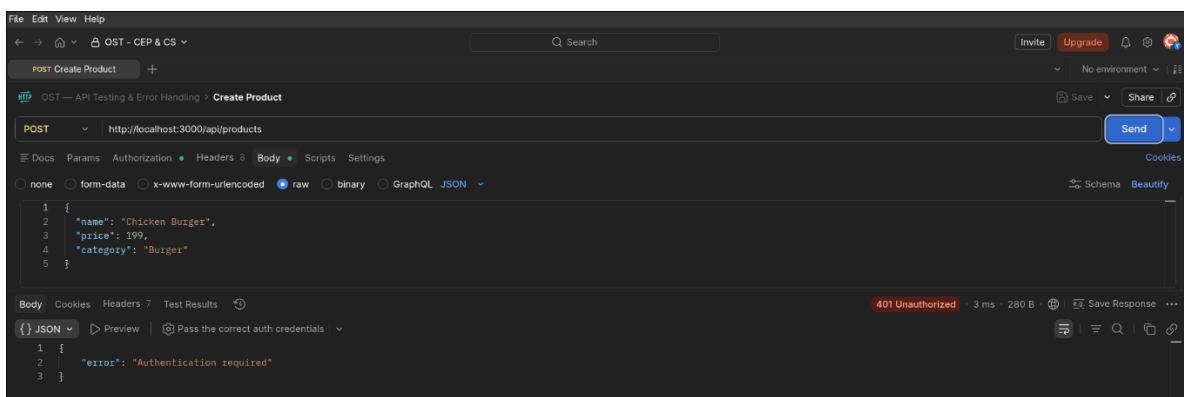


Figure 7: Unauthorized Product Creation Request

COMPLEX ENGINEERING PROBLEM - REPORT

Step 5: Create a Product with Authentication

The same product creation operation was repeated with valid authentication.
The API returned **201 Created**, confirming that the product was successfully created.

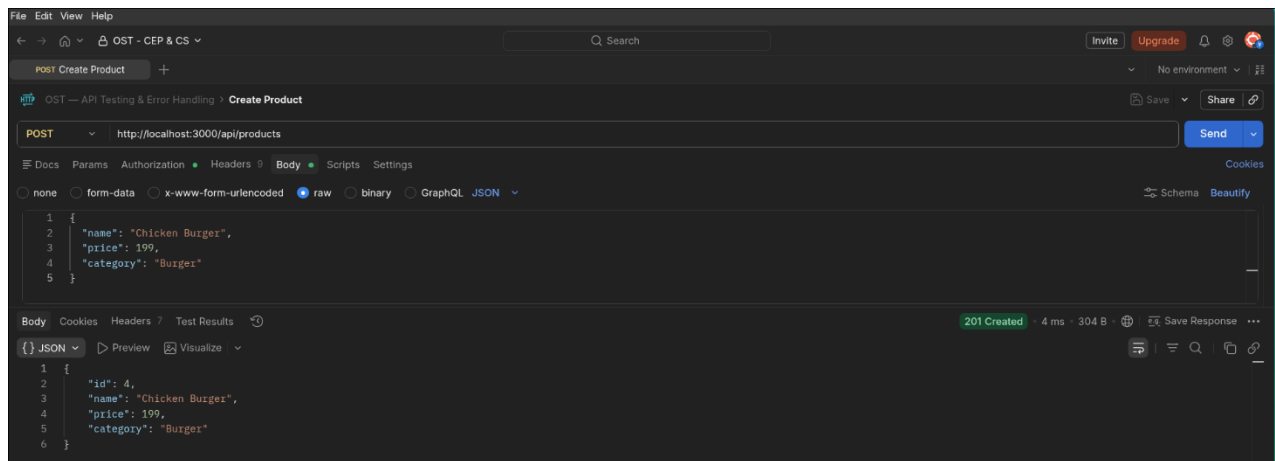


Figure 8: Successful Product Creation with Authentication

Step 6: Verify the Created Product

The product collection was retrieved again to verify that the newly created product was present.
The product was also retrieved directly using its ID. Both requests returned **200 OK**, confirming that the product had been stored and could be retrieved.

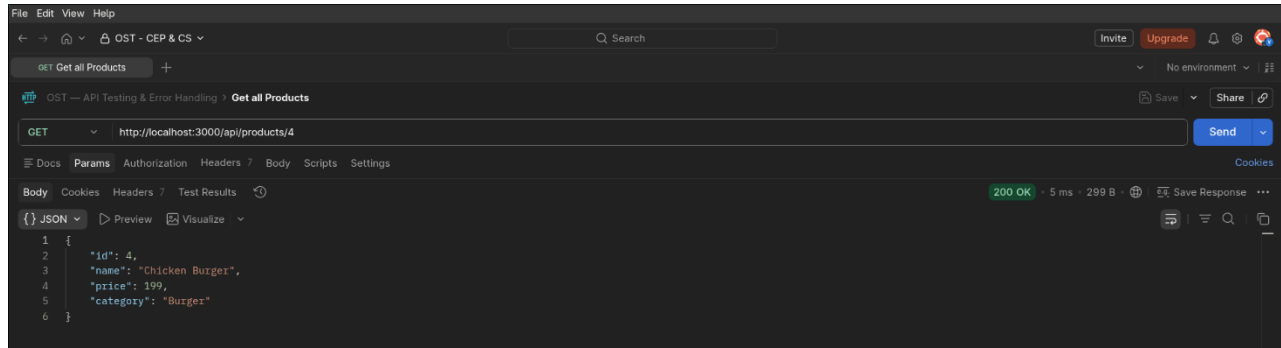


Figure 9: Verification of the Created Product

Step 7: Test Invalid Input

The product creation endpoint was tested with an invalid negative price.
The API returned **400 Bad Request** and an appropriate validation error.

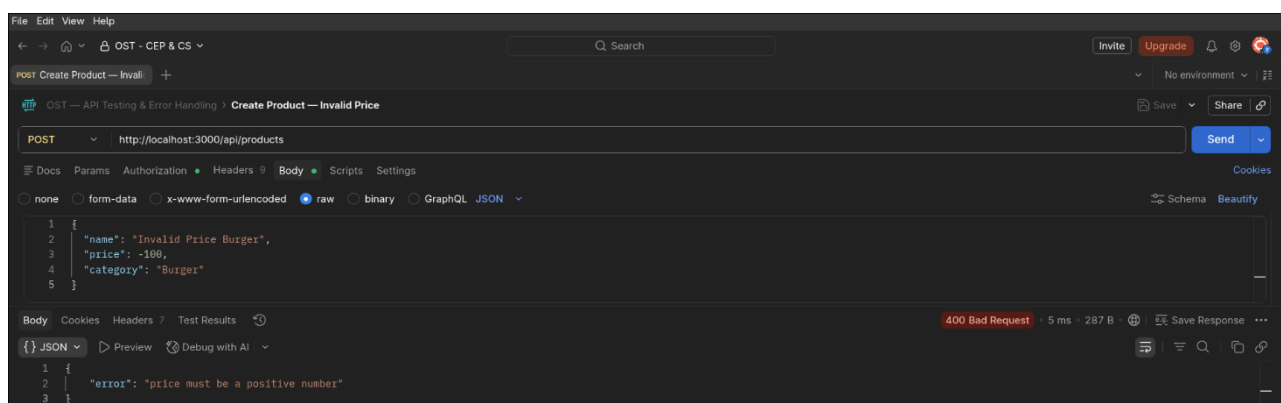


Figure 10: Product Creation with Invalid Price

COMPLEX ENGINEERING PROBLEM - REPORT

Step 8: Test Missing Input

The product creation endpoint was tested after removing the required name field.

The API returned **400 Bad Request**, indicating that the missing required field was detected and rejected.

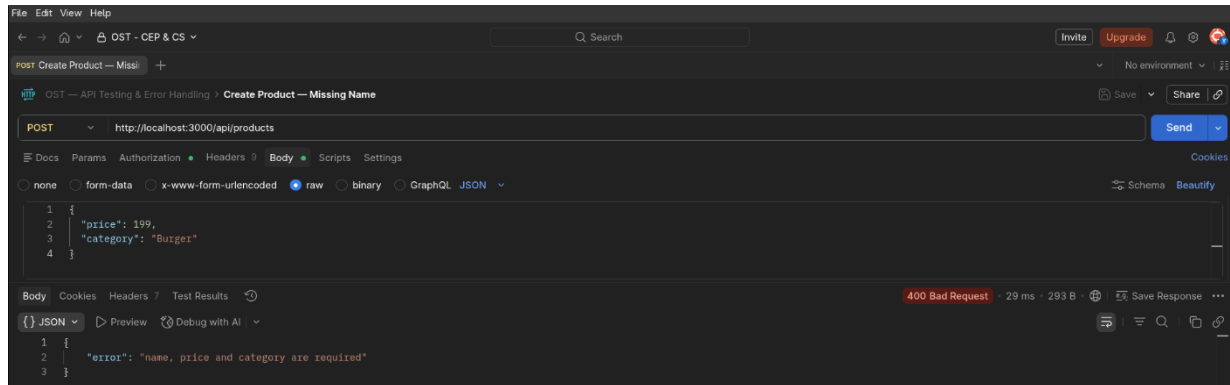


Figure 11: Product Creation with Missing Required Field

Step 9: Update the Product

The existing product was updated using the PUT method: `PUT http://localhost:3000/api/products/4`

The API returned **200 OK**, confirming that the existing product was successfully updated.

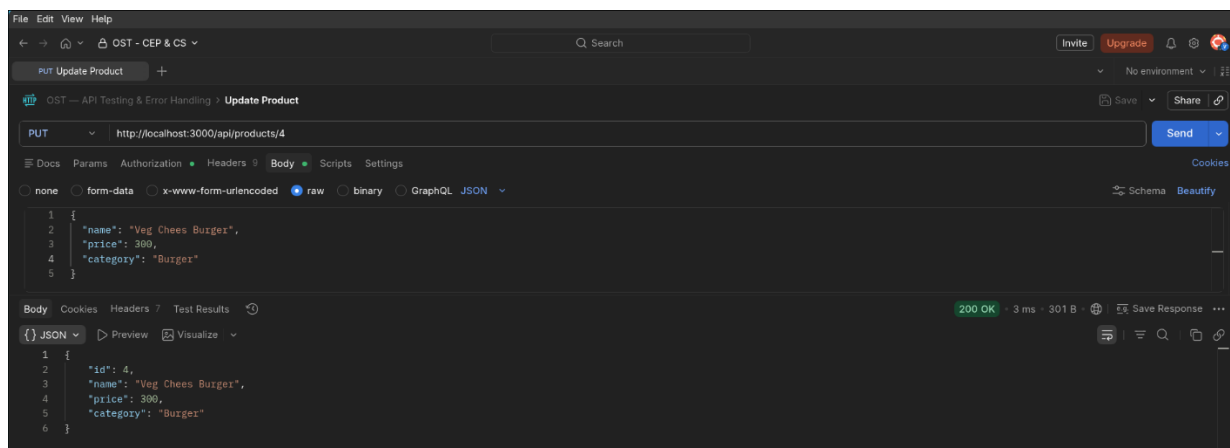


Figure 12: Updating an Existing Product

Step 10: Verify the Updated Product

The updated product was retrieved using its ID.

The API returned **200 OK**, confirming that the updated values had been successfully stored.

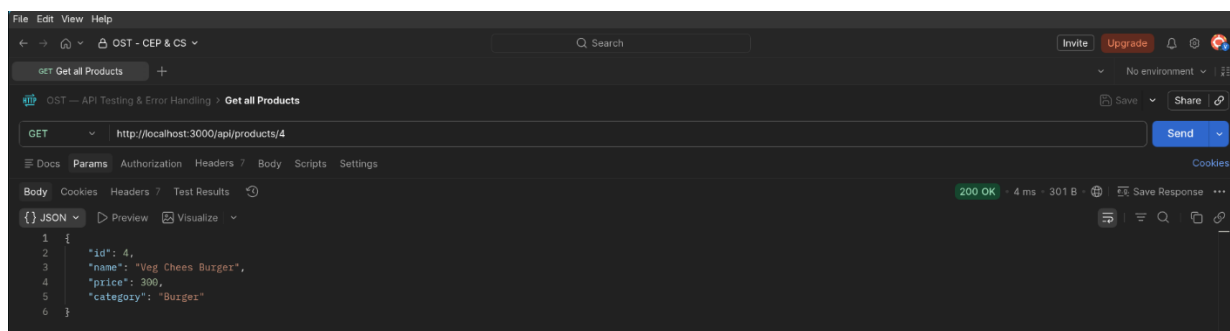


Figure 13: Verification of Updated Product

COMPLEX ENGINEERING PROBLEM - REPORT

Step 11: Delete the Product

The product was deleted using: **DELETE** `http://localhost:3000/api/products/4`

The API returned **200 OK**, confirming that the delete operation was completed successfully.

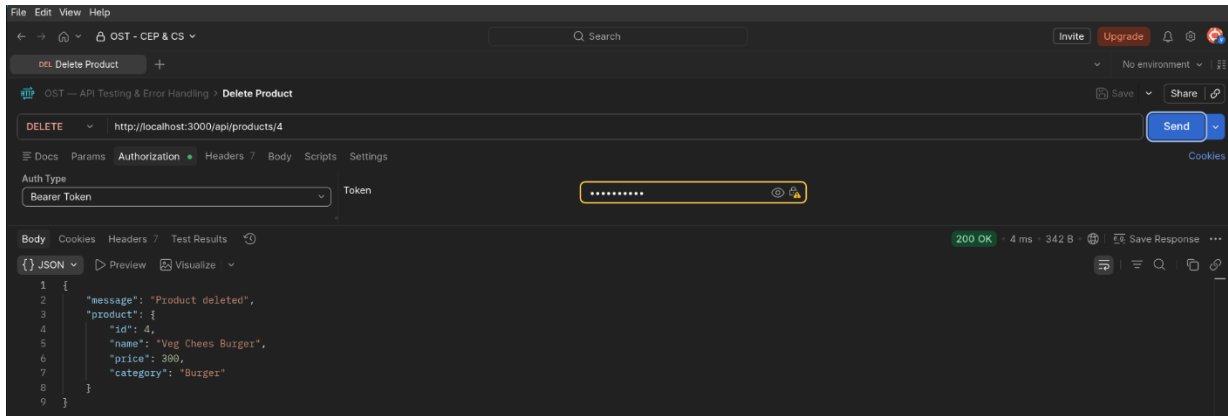


Figure 14: Deleting the Product

Step 12: Verify Deletion

After deletion, the same product ID was requested again.

The API returned **404 Not Found**, confirming that the product was no longer available.

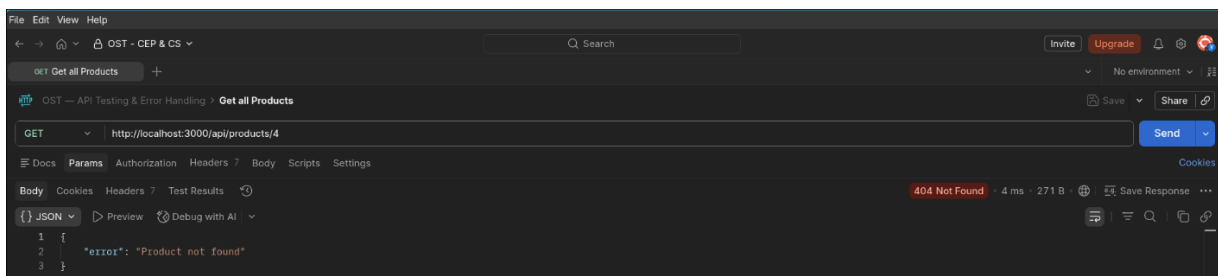


Figure 15: Verification of Product Deletion

COMPLEX ENGINEERING PROBLEM - REPORT

7. Testing and Results

The API was tested using positive and negative test cases. The tests covered authentication, CRUD operations, input validation, unauthorized access, and nonexistent resources.

7.1 Test Result Table

Test / Parameter	Expected Result	Obtained Result
API health check	Server should respond successfully	200 OK
Valid authentication	User should be authenticated	200 OK
Invalid credentials	Request should be rejected	401 Unauthorized
Missing password	Validation error should be returned	400 Bad Request
Get all products	Product collection should be returned	200 OK
Create product without authentication	Request should be rejected	401 Unauthorized
Create product with authentication	Product should be created	201 Created
Retrieve created product	Product should be available	200 OK
Negative product price	Invalid value should be rejected	400 Bad Request
Missing product name	Required field error should be returned	400 Bad Request
Update product	Product should be updated	200 OK
Retrieve updated product	Updated values should be returned	200 OK
Delete product	Product should be deleted	200 OK
Retrieve deleted product	Resource should not be found	404 Not Found

The testing demonstrated that the API returned different HTTP status codes according to the outcome of the request. Successful operations returned 200 OK or 201 Created, while invalid, unauthorized, and nonexistent-resource conditions returned 400 Bad Request, 401 Unauthorized, and 404 Not Found, respectively.

7.2 Response and Error Analysis

The API returned JSON responses for both successful and unsuccessful requests.

For invalid product input, for example, the API returned a meaningful error message indicating that the price must be a positive number.

The observed error conditions included:

- 401 Unauthorized for invalid or missing authentication.
- 400 Bad Request for invalid input and missing required fields.
- 404 Not Found when attempting to retrieve a deleted product.

These responses allow a client application to determine the reason for a failed request and handle the result accordingly.

7.3 Authentication Testing

Authentication testing verified that the API could distinguish between authorized and unauthorized requests. Valid credentials successfully returned an authentication token, while invalid credentials were rejected with 401 Unauthorized. Requests without required authentication were also rejected for protected operations.

7.4 CRUD Testing

The Product API was tested through the complete CRUD lifecycle:

Create: A new product was successfully created using POST.

Read: Products were retrieved using GET.

Update: An existing product was modified using PUT.

Delete: A product was removed using DELETE.

The state of the resource was also verified after creation, update, and deletion. This provided confirmation that the operations produced the expected results.

COMPLEX ENGINEERING PROBLEM - REPORT

7.5 Input Validation Testing

Invalid and incomplete requests were deliberately submitted to test the API's validation behavior. A negative price was rejected with **400 Bad Request**, while a request missing the required product name was also rejected with **400 Bad Request**.

7.6 Recommended Improvements

Based on the testing results, the API can be improved by maintaining consistent validation and error responses for invalid inputs. Error responses should provide clear messages that help client applications identify the reason for failure. Authentication and authorization checks should be applied consistently to all protected endpoints. In a production environment, HTTPS, secure token management, rate limiting, centralized logging, and monitoring should also be implemented.

8. Challenges Faced and Solutions

Challenge	Solution
The API needed to be tested under both successful and unsuccessful conditions.	Separate positive and negative test cases were created in Postman.
Protected API operations required authentication.	Requests were tested both without authentication and with valid authentication credentials.
Successful CRUD operations needed to be verified after execution.	Additional GET requests were used to verify created and updated resources.
Invalid values needed to be checked against API validation rules.	Negative price and missing required fields were tested explicitly.
Deletion needed to be verified rather than assumed from the DELETE response.	The deleted resource was requested again and the resulting 404 Not Found response was checked.

9. Conclusion

- The CRUD-based REST API was successfully tested using Postman with valid, invalid, and missing inputs. Authentication and authorization behavior were verified using valid credentials, invalid credentials, and unauthorized requests.
- The complete Product CRUD lifecycle was tested through GET, POST, PUT, and DELETE operations.
- The API returned appropriate status codes including **200 OK**, **201 Created**, **400 Bad Request**, **401 Unauthorized**, and **404 Not Found**.
- Input validation and JSON error responses were also verified.
- The testing demonstrated the importance of consistent validation, authentication, HTTP status codes, and meaningful error handling in REST APIs.

10. SDG Mapping and Justification

This project can be mapped to SDG 9: Industry, Innovation and Infrastructure because reliable software systems and APIs form an important part of modern digital infrastructure.

The project focuses on improving the reliability and maintainability of an API through systematic testing, validation, authentication checks, and error handling. Proper API testing helps ensure that software services behave predictably when integrated with other applications and systems.

11. References

- [1] Postman, "Postman Documentation," Postman. [Online]. Available: <https://learning.postman.com/docs/>
- [2] IETF, "HTTP Semantics," RFC 9110, Jun. 2022.
- [3] OWASP Foundation, "OWASP API Security Top 10," OWASP Foundation.
- [4] OpenAPI Initiative, "OpenAPI Specification," Linux Foundation.